

CSS3 Styling, Flexbox, Responsive Design



S03

Session Goals



SESSION GOALS

By the end of this session,
you will be able to

- Style modern UI elements using border-radius, box-shadow, and transition for smoother interactions.
- Build flexible layouts using Flexbox properties like display: flex, justify-content, align-items, and flex-direction.
- Create responsive designs using media queries to adapt your layout across mobile, tablet, and desktop.
- Understand and apply key CSS3 properties to enhance user experience and visual appeal.
- Develop a simple responsive web page that looks great on any device.

S01

Topics Covered



Topics Covered

These are the main topics we'll cover today to help you build your first web page

01. Flexbox Layout

- display: flex
- flex-direction
- justify-content
- align-items
- flex-wrap

02. Responsive Design

- Media Queries (min-width, max-width, between ranges)
- Mobile-first approach

02. CSS3 Styling

- border-radius
- box-shadow
- transition

03. Hands-On Tasks

- **Task 1: Flexbox Layout** : Build a simple layout using Flexbox that includes: A header with logo and navigation A main section with 3 cards side by side A footer
- **Task 2: Styling with CSS3** : Style the 3 cards using: border-radius to round corners box-shadow to add depth transition for smooth hover effects 🚀 Add hover effect that slightly enlarges the card.
- **Task 3: Make it Responsive** : Use media queries to: Stack the cards vertically on small screens (max-width: 768px) Reduce font sizes on mobile Adjust padding/margins for better mobile layout

803

Flexbox



FLEXBOX

What is Flexbox?

Flexbox is a layout model in CSS3 that allows you to align, distribute, and space items inside a container – even when their size is unknown or dynamic.

Why use Flexbox?

- *Makes responsive layouts easier.*
- *Eliminates float hacks and complex grid setups.*
- *Powerful alignment controls.*
- *Makes all direct children flex items.*
- *Changes the default layout from vertical (block) to horizontal (row).*

FLEXBOX

Flexbox

Enabling Flexbox :

```
.parent {  
  display: flex;  
}
```

- *Makes all direct children flex items.*
- *Changes the default layout from vertical (block) to horizontal (row).*

FLEXBOX

Flexbox

flex-direction :

```
.container {  
  flex-direction: row / column;  
}
```

- *row : Items are laid out horizontally, from left to right.*
- *column : Items are laid out vertically, from top to bottom.*

FLEXBOX

Flexbox

justify-content :

```
.container {  
  justify-content: flex-start | center | flex-end | space-between | space-around ;  
}
```

- *flex-start* : Items align to the start of the container (left by default).
- *center* : Items are centered in the container.
- *flex-end* : Items align to the end of the container (right by default).
- *space-between* : Items are spaced evenly, with no space at the edges.
- *space-around* : Equal space around each item.

FLEXBOX

Flexbox

align-items :

```
.container {  
  align-items: flex-start / center / flex-end ;  
}
```

- *flex-start* : Items align to the top of the flex container (start of the cross axis).
- *center* : Items are aligned vertically centered within the container.
- *flex-end* : Items align to the bottom of the flex container (end of the cross axis).

FLEXBOX

Flexbox

flex-wrap :

```
.container {  
  flex-wrap: nowrap / wrap;  
}
```

- *nowrap* : All items stay on one single line, even if they overflow.
- *wrap* : Items will wrap to the next line if they don't fit in one row.

FLEXBOX

Flexbox

gap :

```
.container {  
  gap: 20px;  
}
```

- *Adds spacing between items (no need for margins)*
- *Works with row and column gaps*

Media Query



What is Media Query?

A Media Query is a feature in CSS that allows you to apply different styles based on the screen size, device type, or other conditions (like orientation or resolution).

There are 3 common types of media query conditions based on screen width:

- *min-width* : Used when you want to apply styles to larger screens (like tablets or desktops).

```
@media (min-width: 768px) {  
  /* Styles for tablets and up */  
}
```

- *max-width* : Used when targeting smaller screens (like mobile phones).

```
@media (max-width: 768px) {  
  /* Styles for tablets and up */  
}
```

What is Media Query?

A Media Query is a feature in CSS that allows you to apply different styles based on the screen size, device type, or other conditions (like orientation or resolution).

There are 3 common types of media query conditions based on screen width:

- *min-width + max-width* : Useful when you want to style for a specific range (like tablets only).

```
@media (min-width: 600px) and (max-width: 1024px) {
```

```
  /* Styles for tablets */
```

```
}
```


S03

CSS Essentials



CSS Essentials

:hover : A pseudo-class used to apply styles when the user hovers the mouse over an element, Used to create interactive effects like buttons changing color.

```
button:hover {  
    background-color: blue;  
    color: white;  
}
```

opacity : Controls how transparent an element is, 1 = fully visible, 0 = fully transparent

```
img {  
    opacity: 0.5;  
}
```

CSS Essentials

transition : Smoothly animates changes in CSS properties over time

```
element {  
  transition: property duration;  
}
```

border-radius : Rounds the corners of elements

```
element {  
  border-radius: 10px;  
}
```

CSS ESSENTIALS

CSS Essentials

box-shadow : Adds shadow around an element

element {

box-shadow: offset-x offset-y blur-radius color;
}

CSS Essentials

background-image : property is used to set an image as the background of an element (e.g., div, section, body, etc.)

```
selector {  
  background-image: url("image.jpg");  
}
```

Related Properties:

- **background-repeat** : Controls if/how the image repeats (**repeat**, **no-repeat**, **repeat-x**)
- **background-size** : Resizes the image (**auto**, **cover**, **contain**)
- **background-position** : Controls the position (**top**, **center**, **right**, etc.)
- **background-attachment** : Controls scroll behavior (**scroll**, **fixed**)

CSS Essentials

display : Controls how an element is rendered in the layout.

```
p {  
  display: inline;  
}
```

Main values :

- **block** : *Takes full width of the container and always starts on a new line.*
- **inline** : *Sits beside other elements on the same line, width and height cannot be set.*
- **inline-block** : *Behaves like inline (stays in line) but allows you to set width and height Also, margin and padding work fully on all sides*
- **none** : *Controls how scrolling works (e.g., scroll, fixed, none), used to show/hide elements or fix position during scroll.*

CSS Essentials

overflow : controls how content that exceeds the size of its container is handled and displayed.

```
.box {  
  width: 200px;  
  height: 100px;  
  border: 1px solid black;  
  overflow: auto; /* Scrollbar appears only when needed */  
}
```

Main values :

- ***visible***: Content spills out of the container (default).
- ***hidden*** : Excess content is clipped and not visible.
- ***scroll*** : Scrollbars always shown, even if not needed.
- ***auto*** : Scrollbars shown only if content overflows.

CSS Essentials

position : property specifies how an element is positioned in the document.

Main values :

- ***static*** : Default. Element flows normally in the page (no positioning).
- ***relative*** : Positioned relative to its normal position; can be shifted using top, left, right, bottom.
- ***absolute*** : Positioned relative to the nearest positioned ancestor (not static). Removed from normal flow.
- ***fixed*** : Positioned relative to the viewport; stays fixed on screen during scroll. Removed from normal flow.
- ***sticky*** : Switches between relative and fixed depending on scroll position.

Thank you for attending
this presentation!